UNIVERSITY OF SALERNO

SOFTWARE DEPENDABILITY COURSE

Vaporant

Repository: https://github.com/gianfrancobarba/VaporantSD

Project Report

Professor:
Prof.
Dario Di Nucci

Students:
Gianfranco Barba
Std. ID: 0522502025

Francesco Corcione
Std. ID: 0522502027

Luigi Guida
Std. ID: 0522502026

2025/2026

Contents

List of Figures

1 Context and Objectives

The *VaporantSD* project was carried out within the *Software Dependability* course as a refactoring and hardening activity on an existing software system, with the goal of integrating dependability techniques and practices into the development lifecycle.

The work was structured as a coordinated improvement campaign on the overall quality of the application, addressing correctness, security, testability, measurement, and delivery automation.

Vaporant is a Java-based web application built around an e-commerce scenario, providing features for browsing the product catalog, managing a shopping cart, and completing orders.

The system supports different user categories: guest users can browse the product catalog, view item details, manage a cart, and register an account, while registered users can finalize purchases, access their order history, and manage personal information. An administrative role is also supported, enabling the management of products and customer orders through a dedicated interface. Overall, these features define a complete but contained application scope, including user management, transactional workflows, and data persistence.

The main objective was to increase the system's reliability through a set of complementary activities, each supported by tools and repeatable outcomes. In particular, the project aims to:

(i) integrate formal JML specifications and verify them automatically using OpenJML in ESC mode (with the Z3 prover), defining invariants, preconditions, and postconditions on the domain classes;
(ii) reduce maintainability, reliability, and security issues through remediation driven by SonarQube, Snyk, and GitGuardian, including refactoring (e.g., constructor injection and structured logging) and controlled dependency upgrades;
(iii) define a multi-layer testing strategy that combines unit tests (JUnit 5 and Mockito) and web-layer slice tests with MockMvc, keeping execution deterministic and independent from external infrastructure;
(iv) measure and govern quality through automated metrics, including code coverage with JaCoCo (with minimum line and branch coverage thresholds) and mutation testing with PiTest to assess the semantic effectiveness of the test suite;
(v) establish performance baselines through microbenchmarking with JMH on representative components (cart business logic, order-processing workflows, and repository operations);
(vi) make execution reproducible and delivery automatable through Docker and Docker Compose (application + MySQL stack, healthchecks, and startup ordering) and a DevSecOps CI/CD pipeline on GitHub Actions with quality and security gates before publishing the image.

The document organizes these activities into dedicated chapters (formal verification, security/quality remediation, testing, coverage, mutation testing, benchmarking, dockerization, and CI/CD), documenting technical choices, encountered issues, and achieved results.

2 JML Integration and Formal Verification

2.1 Goal

This section describes the process of integrating and formally verifying JML (Java Modeling Language) specifications on the model classes of the `VaporantSD` project. The goal was to formally define and verify class invariants, preconditions, and postconditions on the classes in the `com.vaporant.model` package, using OpenJML in ESC (Extended Static Checking) mode with the Z3 prover.

2.2 Scope

The activity focused on the following model classes:

- `ProductBean.java`
- `Cart.java`
- `OrderBean.java`
- `UserBean.java`
- `AddressBean.java`
- `AddressList.java`

For each class, JML invariants and method specifications were introduced in order to guarantee:

- Non-negativity of prices and quantities.
- Temporal consistency of dates (no future dates where they are not allowed).
- Completeness and consistency of user and address data.
- Synchronization between internal support lists (for example in `AddressList`).

2.3 JML Integration Plan

For `OrderBean`, class invariants were introduced to ensure that the total price field (`prezzoTot`) is never negative and that the purchase date (`dataAcquisto`) is non-null. Temporal constraints on the purchase date are not expressed as class invariants, but as a precondition on the `setDataAcquisto` method, which requires any assigned date to be at most one day in the future with respect to `LocalDate.now()` at run time. The constructor was designed to correctly initialize these fields so that no invariant is violated immediately after object creation. Furthermore, the management of the static order counter was specified and refactored to avoid arithmetic overflow and to improve static verifiability, making counter updates explicit in both code and specifications.

For `UserBean`, the `email` field is constrained by a class invariant to be non-null, while other identifying fields such as `numTelefono` and `codF` are not declared non-null in JML. For these latter fields, possible format and length constraints are enforced at the application level rather than in the specification layer. The fields `nome` and `cognome` are also required to be non-null by invariants. Regarding the `dataNascita` field, the specification enforces non-nullity but does not include a time-dependent invariant such as “date of birth must be in the past”. That temporal constraint is implemented as a run-time check (e.g., in setters or validation logic), in order to avoid expressions based on `LocalDate.now()` in class invariants, which are difficult to verify automatically in ESC mode.

For AddressBean, all address-related fields (via, numCivico, citta, cap, provincia, stato) are constrained by invariants to be non-null. A basic constraint on the length of the postal code (CAP) is also enforced, in line with the business rules of the application domain.

For ProductBean, class invariants enforce non-negativity of price and quantity. Getters are annotated as pure and selected fields are declared spec_public, so that JML can inspect the internal state in the specification layer while still preserving Java encapsulation at the implementation level.

For Cart, invariants ensure that the total price is never negative and that the internal list of products is non-null. Method specifications were added for operations that add and remove products, using preconditions to guarantee that product prices remain non-negative and to support the prover in reasoning about total price updates.

For AddressList, invariants express the synchronization constraints between the different internal lists used to represent addresses. The specification explicitly accounts for possible null values, in order to avoid invariant violations in corner cases where internal lists may be uninitialized or partially initialized.

2.4 JML Implementation Details

The following JML constructs were used systematically across the model classes:

- @public invariant for global class constraints (for example non-negativity and non-nullity).
- @requires, @ensures, and @assignable to specify method behavior in terms of preconditions, postconditions, and modifiable locations.
- The annotation /*@ spec_public @*/ on private fields that appear in specifications, to solve visibility issues and allow OpenJML to verify invariants that refer to internal state.

Moreover, the annotation /*@ skipesc @*/ was systematically adopted for:

- Methods such as toString, getJson, and similar utilities, which are heavy for the prover due to extensive string concatenation.
- Methods with complex loops over mutable collections (e.g., ArrayList), where precise loop invariants would be required and would not provide significant benefits for the project objectives.

2.5 Bug Fixes Identified by OpenJML

During the initial runs of OpenJML, several INVALID and Parsing/Error messages emerged, leading to the following interventions.

2.5.1 Visibility and JML Syntax.

- The syntax of @assignable clauses was corrected, for example by using products instead of products[*] for ArrayList fields, in accordance with the subset of collection notation supported by OpenJML.
- Errors in @assignable clauses inside constructors were fixed, in particular by removing this. qualifiers where they are not allowed.

- Misuses of `nowarn` were removed and replaced with `skipesc`, in line with the OpenJML documentation and recommended practice.

2.5.2 Constructors and Non-Nullity Invariants. In `UserBean`, `AddressBean`, and `OrderBean`, the default constructors initially left some fields as `null`, violating non-nullity invariants. New constructors were introduced (or existing ones were refactored) to initialize these fields with valid default values, thus satisfying the invariants immediately after object creation.

In `OrderBean`, the `dataAcquisto` field was also given a proper default initialization in the constructor, so that the temporal invariant on the purchase date is respected at implementation level.

2.5.3 Explicit Price Calculation and Overflow Prevention. In `Cart`, the management of the total price was made fully explicit in both the implementation and the JML specifications. Compound assignment patterns such as `totale += prezzo` were replaced with explicit intermediate variables (e.g., `double newTotal = prezzoTotale - oldPrice + newPrice`), which are simpler for the Z3 prover to reason about and eliminate the risk of specification-layer arithmetic ambiguities. The `setPrezzoTotale` method enforces a non-negativity guard (`newTotal >= 0 ? newTotal : 0`), ensuring that the invariant `prezzoTotale >= 0` is maintained by construction after every price update.

2.5.4 Null-Safety and Complex Comparisons. In `Cart`, the possibility that `containsProduct` returns `null` was addressed by adding explicit checks, preventing invariant violations during product lookup and removal operations.

String comparisons that were potentially problematic for the prover were replaced with comparisons based on integer codes, which are simpler to handle for SMT-based verification engines such as Z3.

2.6 Verification Script

A shell script named `verify-jml.sh` was used in the project root directory. This script invokes OpenJML in ESC mode on all relevant model classes, using the `target/classes` classpath generated by `mvn compile`. The script can be integrated into CI/CD pipelines to automate the formal verification step.

2.7 Issues and Solutions

The detailed discussion of issues and their solutions for each class is organized in the following subsections (corresponding to Sections 3.1–3.6 of the original report), which are kept unchanged in structure and content:

- **Cart.java**: correction of `@assignable` clauses, use of `skipesc` for methods with loops, and robust handling of the total price and `null` values.
- **OrderBean.java**: refactoring of the constructor (static counter and purchase date) and use of `skipesc` for more complex logic.
- **UserBean.java**: simplification of invariants (removal of time-dependent constraints) and addition of constructors compatible with the specified preconditions.

- **ProductBean.java** and **AddressBean.java**: confirmation of correct specifications and use of `pure` methods and simple numeric invariants.
- **AddressList.java**: management of synchronized lists via invariants, with `skipesc` on methods that update multiple collections in parallel.

2.8 Final Results

2.8.1 Global Statistics. The final verification campaign produced the following global statistics:

- 71 methods formally verified (`Valid`).
- 18 methods marked with `skipesc` and documented with a clear rationale (complex loops, `toString`, constructors with global logic).
- 0 invalid methods, 0 parsing or JML semantic errors, and 0 timeouts.

On average, approximately 79.8% of the methods in the six model classes were fully verified. The verification rate exceeded 90% for `UserBean` and `AddressBean`, thanks to the relatively simple structure of their methods (mostly getters and setters).

2.8.2 Types of Methods Verified. The following categories of methods were successfully verified:

- All getters annotated as `pure` and without side effects.
- All setters with simple preconditions (non-nullity, non-negativity).
- More complex methods (loops over `ArrayList`, synchronization across multiple lists, string concatenation) were marked with `skipesc`, still benefiting from JML type checking even though ESC was disabled for them.

2.9 Emerging Best Practices

From the integration and verification work, the following best practices emerged:

- Prefer *simple and time-stable* invariants (non-nullity, non-negativity) instead of invariants that depend on the current time or external logic.
- Specify `@assignable` clauses precisely, avoiding overly generic patterns such as `\everything` and unsupported forms for collections (e.g., `products[*]` on `ArrayList` fields).
- Use `/*@ skipesc @*/` on methods where the effort of specifying and verifying detailed loop invariants would be disproportionate to the benefits, especially for mutable collections and utility methods such as `toString`.

2.10 Conclusions

The integration of JML into the VaporantSD project led to:

- Executable formal specifications for the main model classes.
- A reduction in logical errors, thanks to explicit class invariants and method preconditions.
- A repeatable automatic verification process, implemented via the `verify-jml.sh` script and suitable for CI/CD integration.

The main limitations concern the difficulty of automatically verifying methods that manipulate complex mutable collections, time-dependent properties, and logic involving global static variables. For these aspects, a combined approach was adopted, using JML together with unit tests and manual code review to ensure the overall reliability of the system.

3 Static Analysis and Security Tools Remediation

3.1 SonarQube Issues Resolution

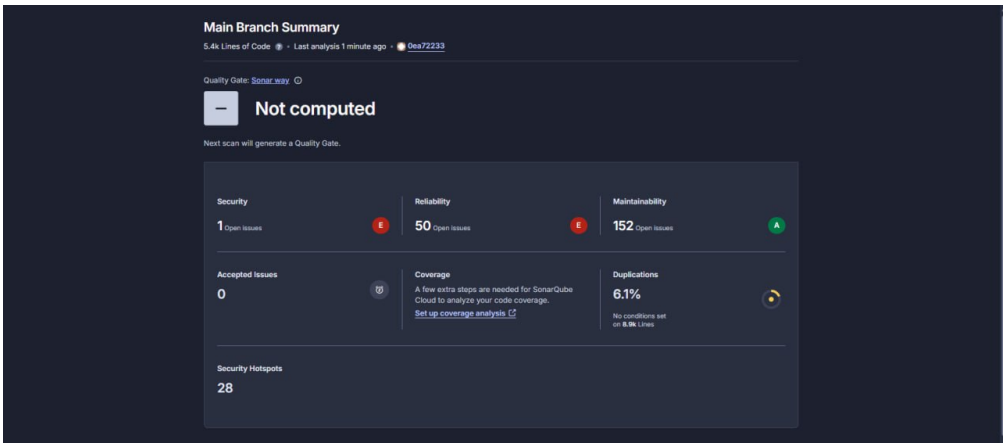
The VaporantSD codebase was scanned with SonarQube to identify issues related to reliability, maintainability, security, and accessibility. The remediation effort focused on addressing all blocking and high-severity issues while improving overall code quality without changing the external behaviour of the application.

A first group of changes concerned dependency injection in Spring components. All occurrences of field injection with `@Autowired` were refactored to constructor injection both in controllers and in data access classes, allowing dependencies to be declared as `final` and making them explicit in constructors. This improves immutability, thread-safety, and testability, and ensures that wiring errors are caught at startup (fail-fast) instead of at runtime.

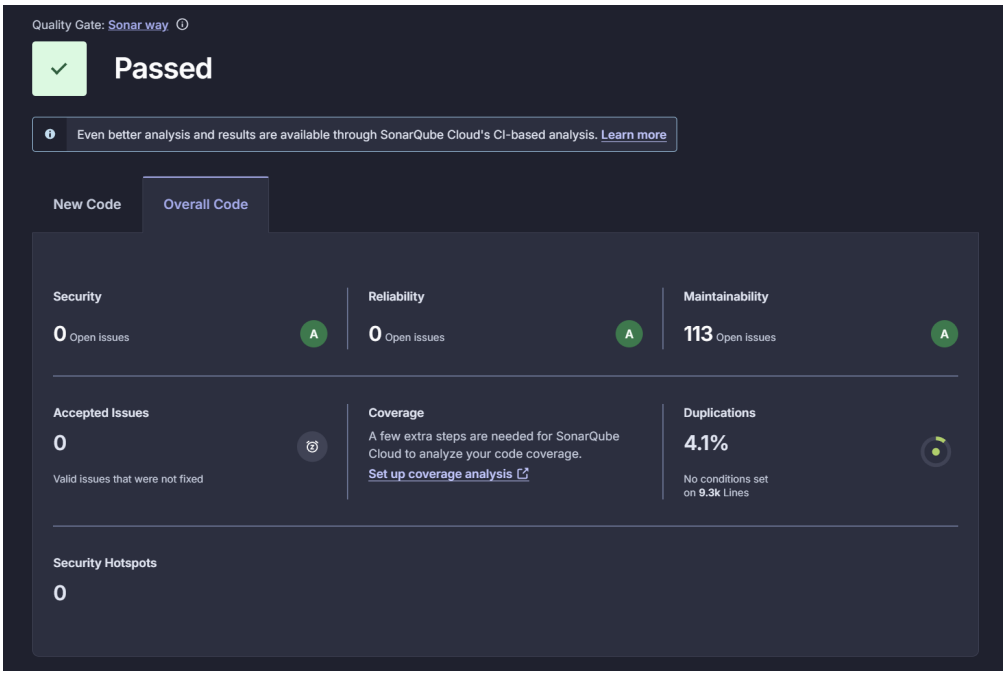
A second group of changes involved logging. Direct uses of `System.out.println` and `printStackTrace()` in error handling paths were replaced with structured logging based on SLF4J and the underlying Spring Boot logging stack. This provides configurable log levels, better performance and formatting, and avoids uncontrolled output of stack traces and potentially sensitive information in production logs.

Several maintainability and duplication issues were addressed by extracting repeated string literals into static constants, especially in SQL queries and table names, in order to follow the DRY principle and reduce the risk of inconsistent changes. Accessibility-related issues detected in JSP views were also fixed by adding missing `lang` attributes on the root `<html>` elements and by properly associating form labels and inputs through the `for` and `id` attributes, improving screen reader support and conformance with WCAG guidelines.

Medium-severity issues at the presentation layer, such as duplicated CSS properties in stylesheets, were fixed by consolidating redundant declarations into a single, consistent definition. Overall, the SonarQube remediation phase eliminated all blocking and high-severity issues and reduced code smells, leading to a codebase that is easier to test, maintain, and extend while respecting established clean code and accessibility practices.



(a) Before remediation



(b) After remediation

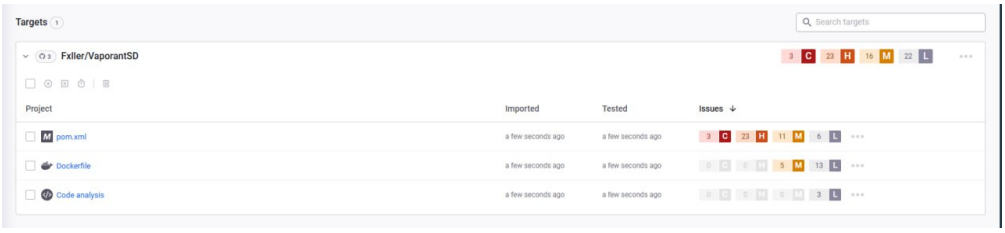
Fig. 3.1. SonarQube results before and after the remediation activities.

3.2 Snyk Vulnerability Remediation

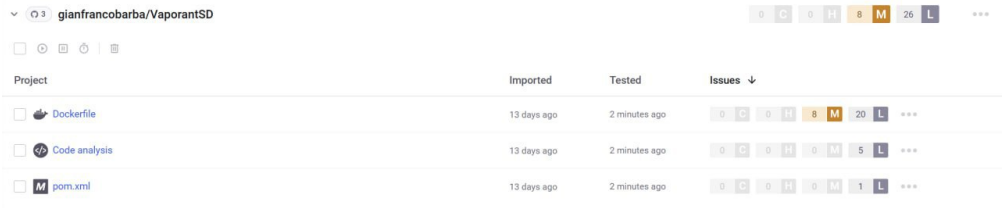
Snyk was used to scan the Maven dependencies of the project and initially reported 26 vulnerabilities (3 critical, 23 high) in transitive libraries pulled in through the `pom.xml`. All 65 underlying CVEs associated with these findings were remediated by a small set of controlled dependency upgrades, without introducing breaking changes.

The main remediation strategy was to update the Spring Boot parent to a newer stable release and to align the embedded Tomcat and Spring Framework versions with secure, supported versions. This automatically pulled in patched versions of several core modules (e.g., `spring-web`, `spring-webmvc`, `spring-beans`, `spring-core`) and logging libraries (`logback-classic`, `logback-core`), eliminating a large number of high-severity issues such as denial-of-service, path traversal, and open redirect vulnerabilities. Where necessary, explicit version properties for Tomcat were introduced to ensure that all embedded components used a non-vulnerable version.

In addition, the MySQL JDBC driver was pinned to a secure release, replacing the vulnerable version inherited from the original Spring Boot BOM. This resolved a remaining access-control vulnerability in the database connectivity layer. After these changes, the project builds successfully with Java 17, the application starts correctly in both local and Dockerized environments, and Snyk reports no remaining critical or high vulnerabilities in the dependency tree.



(a) Before remediation



(b) After remediation

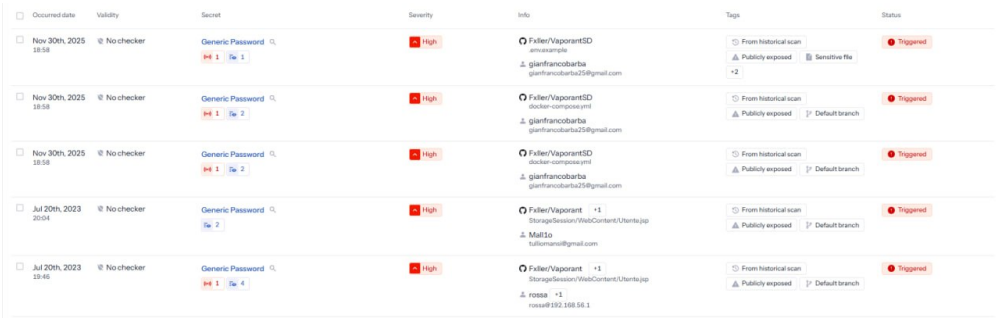
Fig. 3.2. Snyk findings before dependency upgrades and after vulnerability remediation.

3.3 GitGuardian Findings and Secret Management

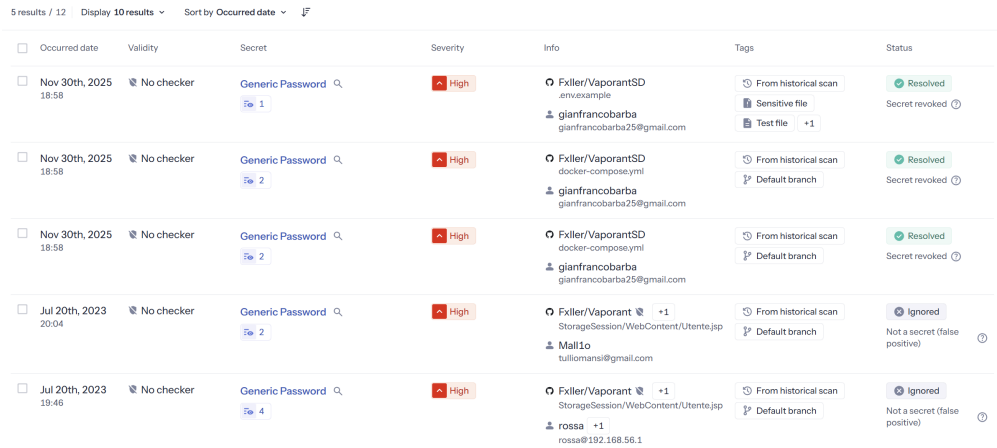
GitGuardian was employed to detect potential secret leaks and credential exposures in the repository. A first set of findings corresponded to false positives on password-related labels in HTML templates, which were reviewed and confirmed as benign, requiring no code changes.

The remaining relevant findings concerned actual secrets present in the codebase and configuration files, in particular plaintext passwords embedded in configuration resources such as `docker-compose` files. These secrets were removed from version-controlled files and externalized into dedicated secret management mechanisms (e.g., environment variables or Docker/Compose secrets), so that sensitive values are no longer stored in the

repository. The application is now configured to read credentials at runtime from the surrounding environment, improving the overall security posture and aligning the project with common best practices for secret management in containerized deployments.



(a) Before remediation



(b) After remediation

Fig. 3.3. GitGuardian detections before secret cleanup and after externalized secret management.

3.4 Overall Approach

Across SonarQube, Snyk, and GitGuardian, the remediation work followed a common methodology: issues were first identified and classified by severity, then manually reviewed to filter out false positives and to understand their impact on the system. For each confirmed issue, the fix consisted in applying widely accepted best practices (e.g., constructor injection in Spring, structured logging, DRY refactoring, dependency upgrades to patched versions, and externalized secrets) while preserving the observable behaviour of the application.

Each set of changes was validated through local builds, automated tests, and application startup checks, and then re-scanned by the corresponding tools to confirm that the issues had been resolved. This iterative process resulted in a codebase that is not only free from the reported issues, but also more robust, maintainable, and secure by design.

3.5 Security Mechanisms in CI/CD

Security checks are not limited to local tooling, but are also integrated into the continuous integration and delivery pipeline via GitHub Actions. The main build workflow (`Build and Analyze`) is triggered on pushes and pull requests to the main development branches and performs a full Maven build with tests, generates JaCoCo coverage reports, uploads coverage to Codecov, and runs a SonarCloud analysis using the previously compiled classes and coverage data. All external services (Codecov and SonarCloud) are configured through GitHub encrypted secrets, so that authentication tokens and project identifiers are never hard-coded in the repository.

In addition to quality and coverage checks, a dedicated `Security Pipeline` workflow is responsible for automated security scanning. This workflow is triggered automatically whenever the main build completes successfully, or can be started manually via `workflow_dispatch`. It runs GitGuardian to detect potential secret leaks, Snyk SAST (`snyk code test`) to scan the source code for vulnerabilities, and Snyk dependency scans on the Maven project with a high-severity threshold, ensuring that both the application code and its transitive dependencies are continuously monitored for security issues.

The security workflow also builds the Docker image of the application and runs a Snyk container scan on the resulting image, checking the runtime environment and base layers for known vulnerabilities. All Snyk commands use the `SNYK_TOKEN` stored as a GitHub secret, and the pipeline is configured so that security scans run only after a successful build, avoiding noise from failing or unstable code. Overall, this CI/CD setup enforces a security gate that combines static analysis, dependency auditing, secret detection, and container scanning, providing continuous feedback on the security posture of the project.

4 Testing Strategy and Methodological Framework

The testing strategy adopted for the *Vaporant* project is designed as a **multi-layer** system, where unit tests and web-layer integration tests cooperate to validate the core business logic and HTTP flows, while keeping executions fast and deterministic.

4.1 Overall Testing Strategy

The primary objective is to ensure that the critical parts of the application (business logic, state management, data access, and the web layer) are protected by automated tests that are granular enough to detect regressions, without depending on external infrastructure such as real databases or application servers.

To achieve this, the test suite combines:

- **Unit tests** on the model, repositories with non-trivial logic, and utility components, based on JUnit 5 and Mockito, completely independent of the Spring container.
- **Web-layer integration tests (slice testing with MockMvc)** on controllers, executed within the Spring Boot test context using `@WebMvcTest` and `MockMvc`, with DAOs provided as mocks.

This layout effectively realizes a *test pyramid*: a broad base of unit tests, an intermediate layer of web slice tests, and coverage/mutation metrics configured in `pom.xml` as quality gates guiding overall test quality.

4.2 Test Design Approach

The design of test cases follows a systematic, scenario-based approach grounded in well-known testing principles:

- **Equivalence partitioning (pragmatic)**: for each feature, input values are grouped into nominal (valid) cases, invalid inputs, and error scenarios. Typical examples include login with correct/incorrect credentials, different user types, valid/invalid addresses, and null or empty parameters in registration flows.
- **Boundary value analysis**: tests explicitly target boundary conditions, especially in cart and quantity management (quantity equal to stock, exceeding stock, zero, negative values) and in list index handling for address collections.
- **Scenario coverage**:
  - *Happy path*: normal flows with valid inputs and expected successful behaviour (e.g., successful login, adding a product to the cart, successful user persistence).
  - *Edge cases*: limit situations such as empty carts, quantities at stock limits, optional parameters omitted, and collections containing null elements.
  - *Error handling*: scenarios where dependent components (e.g., DAOs) throw exceptions—in particular `SQLException`—and the controller or repository is expected to handle them gracefully.

Parameterized tests (`@ParameterizedTest` with `@CsvSource` and `@ValueSource`) are used to exercise many variations of the same logical scenario from a single test definition, for example to verify redirects based on the user type in the login flow or to validate multiple invalid email patterns during registration. Overall, while the suite does not formalize Category Partition Testing with explicit partition tables, it consistently applies

the underlying ideas of equivalence partitioning and boundary analysis to the most sensitive functionalities.

4.3 Unit Tests: Model, Repository and Utility Layers

Unit tests focus on classes that contain non-trivial logic and can be exercised in isolation.

Model layer. The model layer is tested through pure JUnit 5 tests:

- `CartTest` verifies the cart management logic: adding and removing products, updating quantities, computing the total price with decimal rounding, protecting against negative prices, and handling null product references.
- `AddressListTest` covers the construction of the address list, retrieval from the underlying DAO, `SQLException` handling, valid/invalid index management, and JSON serialization of the internal collection.

These tests rely solely on JUnit 5 (Jupiter API/Engine 5.12.2 and Params 5.11.3 for parameterized tests) and standard assertions, without starting any Spring container.

Repository layer with business logic. For repositories with non-trivial behaviour, unit tests use Mockito to mock JDBC dependencies:

- `UserDaoImplTest` exercises the main DAO through a mocked chain `DataSource` → `Connection` → `PreparedStatement` → `ResultSet`, validating correct bean population, SQL parameter binding, "not found" cases, `SQLException` handling, and proper closing of JDBC resources.
- `AddressScriptTest` tests the wrapping and data transformation logic implemented by `AddressScript`, including constructors, getters/setters, string formatting, and edge cases on identifiers and addresses.

These tests are executed with JUnit 5 and Mockito 5.17.0 only, without interacting with a real database, thus privileging isolation and fine-grained control over execution paths.

Utility components. The `DataSourceUtilTest` class validates the behaviour of the utility responsible for returning the application `DataSource` from the Spring context, using mocks for `ApplicationContext` and `DataSource` to test caching, context handling, and null safety. In all these cases, the strategy is that of *classical unit testing*: one unit at a time, external dependencies replaced by mocks, no container, very fast execution and high observability of the internal behaviour.

4.4 Web-Layer Integration Tests with MockMvc

Controllers represent the HTTP boundary of the application and are tested via **web-layer integration tests** based on `@WebMvcTest` and `MockMvc`. These tests:

- Start a specialized *Spring Test context for the web layer*, loading the controller under test, the MVC infrastructure (dispatcher, parameter conversion, session management), and the components required to simulate HTTP requests.
- Use `MockMvc` to build requests (e.g., `post("/login")`, `get("/cart")`), set parameters, session attributes and payloads, and to assert HTTP status codes, redirects, JSON content, and response content types.

- Inject DAOs as `@MockitoBean` so that data access logic does not hit any real database and responses can be precisely controlled via Mockito stubbing.

This setup enables end-to-end verification of the web layer behaviour (routing, parameter binding, session management, authentication flows, forwards/redirects) without deploying a full application server, keeping tests both realistic and efficient.

4.5 Test Organization and Naming Conventions

Test organization follows a regular and predictable structure:

- One test class per significant production class, ensuring a clear mapping between production and test code. Representative examples include:
  - `CartTest`, `AddressListTest`, `UserDaoImplTest`,
  - `AddressScriptTest`, `DataSourceUtilTest`,
  - `LoginControlTest`, `CartControlTest`, etc.
- Within each test class:
  - class-level annotations (e.g., `@WebMvcTest`).
  - Extension bindings (e.g., `@ExtendWith(MockitoExtension.class)`).
  - fields annotated with `@Autowired`, `@Mock`, `@MockitoBean`, `@InjectMocks`,
  - a `@BeforeEach` method for per-test initialization,
  - groups of test methods organized by functionality (happy path, edge cases, error handling).

The naming convention for test methods follows a three-part structure: method, scenario, and expected behavior. This makes the intent of each test immediately understandable. Typical examples are listed below:

- `addProduct_newProduct_addsToCart`
- `addProduct_quantityEqualsStock_doesNotIncrement`
- `testLoginWithNullEmail`
- `testModifyEmailUnitVerifyVoidCalls`

Parameterized tests additionally incorporate scenario details through `@CsvSource` and `@ValueSource`, improving traceability of the covered variations directly from the test names and reports.

5 Code Coverage with JaCoCo

Code coverage analysis in the Vaporant project is based on JaCoCo 0.8.12, integrated into the Maven build to automatically measure how much code is actually exercised by tests and to enforce minimum thresholds as a quality gate.

5.1 JaCoCo configuration and quality gate

The `jacoco-maven-plugin` is configured with three distinct operational phases:

- **`prepare-agent`**: enables the JaCoCo agent before test execution, instrumenting the bytecode at runtime.
- **`report`** (`test` phase): generates coverage reports in HTML, XML, and CSV under `target/site/jacoco/`.
- **`check`** (`verify` phase): applies the configured thresholds and fails the build if they are not met.

Thresholds are defined at `BUNDLE` level (the whole project):

- Line coverage $\geq 80\%$.
- Branch coverage $\geq 80\%$.

As a result, a `mvn verify` execution fails if global line or branch coverage drops below 80%, turning coverage into an effective quality constraint rather than a mere informational indicator.

5.2 Inclusion/exclusion policy

To prevent classes with little or no logic from artificially affecting the metrics, JaCoCo explicitly excludes some categories of sources:

- **Plain beans** (getters/setters only) in the `model` package:
  - `AddressBean`, `ContenutoBean`, `OrderBean`,
  - `ProductBean`, `UserBean`.
- **Entry point and simple DAOs**:
  - `VaporantApplication` (Spring Boot startup class).
  - In `repository`: `AddressDaoImpl`, `ContenutoDaoImpl`,
  - `OrderDaoImpl`, `ProductModelDM` (simple CRUD).

Coverage analysis therefore focuses on:

- all web controllers,
- the model classes that contain logic (`Cart`, `AddressList`),
- repositories with non-trivial logic (`UserDaoImpl`, `AddressScript`),
- the utility class `DataSourceUtil`.

The exclusions configured in JaCoCo are aligned with those of PiTest, ensuring consistency between traditional coverage and mutation testing.

5.3 Global coverage metrics

The latest JaCoCo execution on the bundle shows values well above the configured thresholds:

- Instruction coverage: 96%

- Line coverage: 96%
- Branch coverage: 87%
- Complexity coverage: 89%
- Method coverage: 100%
- Class coverage: 100%

vaporant

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.vaporant.controller		95%		89%	11	94	20	349	1	45	0	13
com.vaporant.repository		98%		80%	9	40	3	155	0	15	0	2
com.vaporant.model		99%		94%	2	36	2	76	0	19	0	2
com.vaporant.util		100%		100%	0	5	0	8	0	4	0	1
Total	70 of 2.070	96%	22 of 182	87%	22	175	25	588	1	83	0	18

Fig. 5.1. JaCoCo global coverage report

All classes and all methods within the considered scope are therefore exercised at least once by the tests, and almost all lines and branches are traversed during the suite execution.

5.4 Coverage per layer (package breakdown)

5.4.1 Controllers (com.vaporant.controller). For the controller package, JaCoCo reports:

- Instruction coverage: 95% (1,140/1,196).
- Line coverage: 94% (336/355).
- Branch coverage: 90% (85/94).
- Methods: 100%, Classes: 100%.

In the class-level details, most controllers reach 100% line coverage. Representative examples are `HomeControl`, `CartControl`, `AddressControl`, `LogoutControl`, `FatturaControl`, `SearchBarControl`, and `CustomErrorController`.

Three controllers, however, show small uncovered areas:

- `ModifyControl`: 83% line coverage, 100% branch coverage (24 instructions and 9 lines missed).
- `OrderControl`: 85% line coverage, 70% branch coverage (20 instructions and 6 lines missed; 30% of branches not exercised).
- `ProductControl`: 91% line coverage, 83% branch coverage (12 instructions and 4 lines missed).

The uncovered branches are mainly associated with rare edge cases: specific error conditions, complex validations.

5.4.2 Model (Cart, AddressList). For `com.vaporant.model`, limited to the included business classes (`Cart` and `AddressList`), JaCoCo indicates:

- Instruction coverage: 99% (344/347).
- Line coverage: 97% (76/78).
- Branch coverage: 94% (32/34).
- Methods: 100%, Classes: 100%.

In detail:

- `Cart`: 100% line coverage and 92% branch coverage (2 branches missed out of 26), with all meaningful instructions exercised by the tests.
- `AddressList`: 98% instruction coverage, 100% branch coverage, with 3 missed instructions but no logical branch left unexplored.

This confirms an almost complete coverage of the business logic for the cart and the address-list management, consistent with the excellent results also observed in mutation testing.

5.4.3 Repository (UserDaoImpl, AddressScript). For `com.vaporant.repository` (only on the classes included in JaCoCo and PiTest, i.e. `UserDaoImpl` and `AddressScript`):

- Instruction coverage: 98%.
- Line coverage: 98%.
- Branch coverage: 80% (40 total branches, 10 missed).
- Methods: 100%, Classes: 100%.

In detail:

- `AddressScript` is fully covered (100% instruction coverage, no branches).
- `UserDaoImpl` has 98% covered instructions but only 80% branch coverage (10 branches missed out of 50), mainly due to complex JDBC error-handling paths: catch branches, fallback blocks, and alternative recovery paths that current tests never execute.

This situation is consistent with the picture provided by mutation testing, which identifies the repository layer as a priority area for expanding error-handling tests.

5.4.4 Util (DataSourceUtil). For the util package (`com.vaporant.util`), which contains `DataSourceUtil`, JaCoCo reports:

- Instruction coverage: 100%.
- Line coverage: 100%.
- Branch coverage: 100%.
- Methods: 100%, Classes: 100%.

All branches in `DataSourceUtil`'s logic (ApplicationContext handling, DataSource retrieval and caching, null handling) are exercised by at least one test, with no missed instructions or branches.

5.5 Qualitative analysis of uncovered parts

Uncovered code portions are numerically very limited (24 lines and 21 branches out of 575 lines and 201 total branches), but they are concentrated in points that matter for robustness:

- In the most complex controllers, missed lines and branches correspond to specific parameter combinations (`null`/empty, alternative flows) and rarer validation conditions not yet covered by dedicated tests. The classes most affected are `ModifyControl`, `OrderControl`, and `ProductControl`.
- In `UserDaoImpl`, missed branches are linked to JDBC error-handling paths and fallback branches handling `SQLException` or connection failures: paths implemented for robustness, but not yet exercised by automated tests.

- In the model, the few missed branches (in particular the 2 uncovered branches in `Cart`) are associated with very fine boundary cases, also highlighted by the small number of surviving mutants in that area.

No structural issues or entire blocks of untested logic emerge from the reports: gaps mainly concern rare scenarios and advanced error-handling paths.

5.6 Overall assessment

Overall, JaCoCo results place the Vaporant test suite in a very high quality band:

- The global thresholds (80% line and 80% branch) are exceeded by significant margins (96% line, 88% branch).
- All classes and all methods within the analysed scope are covered at least once.
- The most critical layers (controller, model, complex repository, util) show line coverage between 94% and 100%, with the few missing branches concentrated in error scenarios or less frequent parameter combinations.

In combination with PiTest, JaCoCo is used not just to "hit a number", but as a structural control tool: it ensures that the suite exercises almost all possible paths through the code, while mutation testing verifies that those paths are protected by effective assertions that can detect real logical defects.

6 Mutation Testing with PiTest

While code coverage indicates which parts of the code are executed by tests, mutation testing evaluates their effectiveness in detecting logical defects. In the Vaporant project, mutation testing is implemented using PiTest (`pitest-maven` 1.17.0 with the JUnit 5 plugin), and acts as a true quality gate over the test suite rather than as a mere reporting tool.

6.1 Methodology and configuration

The core idea of mutation testing is to introduce artificial defects (mutants) into the bytecode and observe whether at least one test fails; if a test fails, the mutant is considered *killed*, otherwise it is *survived* and may indicate a weakness in the tests for that code region.

6.2 PiTest setup

The PiTest configuration in `pom.xml` is fully aligned with the generated reports:

- **Version**: PiTest 1.17.0 with `pitest-junit5-plugin` 1.2.3 for native JUnit 5 support.
- **Target classes** (`targetClasses`): all classes in the controller, repository and util packages, plus the business classes `Cart` and `AddressList` in the model package, for a total of 18 classes.
- **Target tests** (`targetTests`): the corresponding 18 test classes (13 controller tests, 2 repository tests, 1 util test, 2 model tests), with a clear 1:1 mapping between mutated classes and test files.
- **Execution settings**: 4 parallel threads and a timeout constant of 3000 ms to avoid hangs caused by pathological mutations (for example infinite loops).
- **Output**: HTML and XML reports, in particular `target/pit-reports/index.html` and `mutations.xml`.
- **Quality gates**:
  - `mutationThreshold` = 50: minimum global mutation score (killed / total mutations).
  - `coverageThreshold` = 80: minimum mutation coverage on covered mutants (also referred to as test strength, i.e. killed / covered mutations).

The configuration does not explicitly specify a `<mutators>` tag, so PiTest applies the standard `DEFAULTS` mutator profile (default behaviour since version 1.0+). This is precisely the set of mutators reported in the analysis documents.

6.3 Mutators

The default mutator profile includes the operators that are most critical for business logic:

- **CONDITIONALS_BOUNDARY**: modifies boundary comparisons (for example, < to <=, >= to >) to stress edge conditions.
- **NEGATE_CONDITIONALS**: negates conditional logic (== to `!=`, < to >=, etc.).
- **MATH**: alters arithmetic operators (for example, + to -, * to /).
- **VOID_METHOD_CALLS**: removes calls to void methods to assess the impact of side effects (logging, state updates, resource cleanup).
- **RETURN_VALS**: changes return values (inverted booleans, constants instead of computed values, `null` or empty collections).

- **Other minor mutators**: such as those acting on increments/decrements or empty returns, useful for stressing less obvious parts of the code.

The test suite is explicitly designed with these mutators in mind: precise numerical assertions (for example, on cart totals) aim at killing **MATH** mutations, while extensive use of Mockito's `verify()` on void methods is intended to kill **VOID_METHOD_CALLS** mutations.

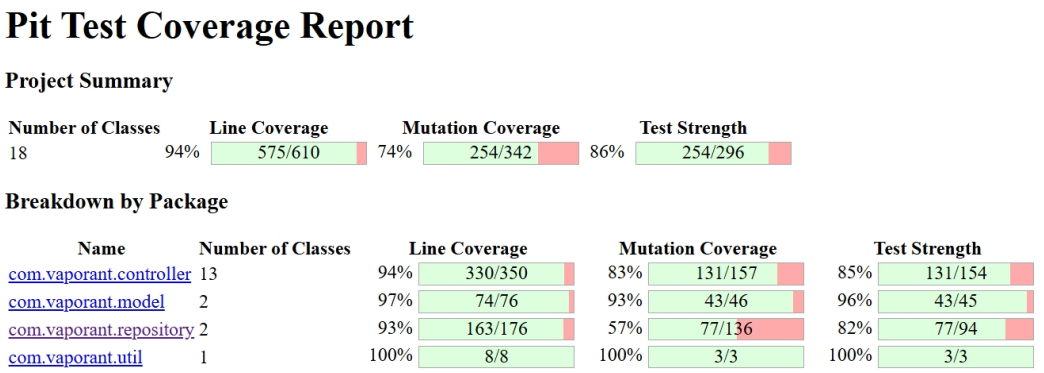
6.4 Global mutation metrics

The final PiTest run produces a total of 342 mutations across the 18 target classes:

- Total mutations: 342.
- Killed mutations: 254.
- Survived mutations: 88.
- No-coverage mutations: 45 (mutations in code that is never executed by any test).

From these figures, PiTest computes the following global metrics:

- Mutation score (killed / total): 74%, above the configured 50% threshold.
- Mutation coverage / Test strength (killed / covered): 86% (254/296), above the 80% threshold.
- Line coverage on mutated classes: 94%, consistent with the JaCoCo analysis over the same code.



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
18	94%	575/610	74%	254/342	86%	254/296

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.vaporant.controller	13	94%	330/350	83%	131/157	85%	131/154
com.vaporant.model	2	97%	74/76	93%	43/46	96%	43/45
com.vaporant.repository	2	93%	163/176	57%	77/136	82%	77/94
com.vaporant.util	1	100%	8/8	100%	3/3	100%	3/3

Fig. 6.1. PiTest global mutation report

Both PiTest quality gates are therefore satisfied: the suite not only kills a large majority of mutants overall, but is especially effective in regions that tests actually execute.

6.5 Results per application layer

The breakdown per package reveals clear differences between layers.

6.5.1 Controllers (com.vaporant.controller).

- Line coverage (PiTest): 94%.
- Mutation score: 85%
- Test strength: 86%

This package shows the best balance between coverage and mutant killing: controllers are thoroughly exercised via `@WebMvcTest` and MockMvc, and the combination of content assertions and `verify()` calls on session and HTTP response (status, redirects, content type) kills most mutants, including many **VOID_METHOD_CALLS** mutations.

6.5.2 Model (Cart, AddressList).

- Line coverage: 97%.
- Mutation score: 93%.
- Test strength: 96%.

The model layer is the strongest part of the suite: the cart and address list logic is almost fully covered, and almost all mutants are killed. In particular, the **MATH** mutator is completely killed, thanks to precise assertions on totals, rounding and composite price calculations, and most boundary and conditional mutations are detected by explicit tests on quantity thresholds and index boundaries.

6.5.3 Repository (UserDaoImpl, AddressScript).

- Line coverage: 93%.
- Mutation score: 57%.
- Test strength: 82%.

The repository layer is the weakest area in mutation terms. Many mutants survive, especially in JDBC error-handling paths and in **VOID_METHOD_CALLS** mutations that remove calls to `close()` on `Connection`, `PreparedStatement` or `ResultSet` inside certain catch or cleanup branches. Several of these branches are either never executed (*NO_COVERAGE*) or executed without explicit `verify()` assertions on resource cleanup, which prevents PiTest from detecting the lost side effects.

6.5.4 Utility (DataSourceUtil).

- Line coverage: 100%.
- Mutation score: 100%.
- Test strength: 100%.

Despite its small size, the util package is fully covered and all mutants are killed. Tests explicitly validate the behaviour of `DataSourceUtil`, including void calls and internal conditions related to the Spring application context.

6.6 Mutator breakdown and emerging patterns

The analysis of `mutations.xml` also highlights patterns per mutator type, with qualitative mapping to packages.

VOID_METHOD_CALLS *(≈ 110 mutations, ≈ 80% killed).* This is the most frequent mutator (roughly one third of all mutations) and the one that best exposes differences between layers.

- In controllers, the combination of MockMvc and `verify()` on void methods (such as `setStatus`, `setContentType`, session handling) yields a very high kill rate: most mutants that remove such calls are killed by the tests.

- In the repository layer, however, error-handling and cleanup paths are less tested: mutations removing `close()` calls in certain catch or finally blocks often appear as *SURVIVED* or *NO_COVERAGE*, because there are no tests that force those error conditions and assert resource cleanup under failure.

MATH. Globally, **MATH** mutations are almost entirely killed.

This is mainly due to `CartTest`, which relies on exact numerical assertions (including decimal rounding and multi-product totals); any alteration of arithmetic operations in the cart logic is immediately detected by these tests.

CONDITIONALS_BOUNDARY *and* ***NEGATE_CONDITIONALS***. Most conditional mutations are killed thanks to explicit boundary tests and scenario-based tests on parameters, quantities and internal states.

A subset of mutants survives in areas where conditions are particularly complex, especially in some controllers (`ModifyControl`, `OrderControl`, `ProductControl`) and in certain recovery branches in the repository that are not exercised by the current tests.

RETURN_VALS *and related mutators.* In the model layer, tests typically assert exact return values, so most **RETURN_VALS** mutations are killed.

In the repository layer, some **RETURN_VALS** mutations survive in methods whose return values indicate error or special states, when tests only check for non-null results or absence of exceptions without verifying the semantic content of the returned value.

6.7 Assessment of robustness

Overall, mutation testing confirms that the Vaporant test suite achieves a high robustness level:

- A 75% mutation score and 86% test strength, both above the configured thresholds (50% and 80%).
- Model and controller layers with excellent results (93% and 85% mutation score respectively) and very few surviving mutants.
- A repository layer with good traditional coverage but lower mutation score (57%), correctly identified by PiTest as the primary candidate for future strengthening, especially in JDBC error-handling paths.
- A utility package fully covered and with all mutants killed.

In this configuration, PiTest is not just a reporting tool but an effective quality gate: it forces tests to be more than simple coverage exercises and drives them towards assertions and verifications that can actually detect real logical defects and regressions.

7 JMH Microbenchmarking

7.1 Goals and Rationale

The VaporantSD project includes performance-critical components such as shopping cart management, repository methods with `synchronized` access, and multi-step order processing workflows. To obtain reliable performance measurements for these components, a dedicated microbenchmarking suite was implemented using the Java Microbenchmark Harness (JMH). The benchmarks were designed to provide quantitative baselines for future optimizations and to validate that recent refactorings did not introduce performance regressions.

Compared to ad-hoc timing based on manual measurements or simple `System.nanoTime()` calls, JMH offers a controlled environment with automatic warmup phases, configurable measurement modes (throughput and average time), and statistical aggregation over multiple iterations. This makes the results more robust and comparable across runs and configurations, especially when evaluating micro-level operations such as individual DAO methods or cart updates.

7.2 Benchmarked Components

The benchmark suite focuses on three main layers of the application: business logic (cart), repository layer, and end-to-end order processing workflows. Components were selected according to four criteria: frequency of use in typical HTTP requests, algorithmic complexity (loops, collection operations, database I/O), potential overhead sources (synchronization, sorting, joins), and business criticality with respect to user-perceived latency.

7.2.1 CartBenchmark: Business Logic Layer. The `CartBenchmark` class exercises the `Cart` component, which implements core in-memory shopping cart operations. The benchmark covers typical cart operations such as adding products, removing products, updating quantities, querying the current cart size, and executing a full cart lifecycle (add, update, delete). Each benchmark method is executed for different cart sizes (e.g., small, medium, and larger carts) to observe how performance scales with the number of items.

For each operation and cart size, JMH is run in both throughput and average time modes, resulting in dozens of benchmark variations that characterize the cost of constant-time operations (such as `getCartSize` and hash-based lookups) versus operations that depend linearly on cart size (such as removals from `ArrayList`). The outcome is a set of baselines showing that cart operations remain extremely fast even for larger carts, and that the in-memory business logic is not a bottleneck for the application.

7.2.2 OrderProcessingBenchmark: Workflow Layer. The `OrderProcessingBenchmark` class targets the workflow that handles order creation and persistence, involving components such as `OrderDaoImpl`, `ContenutoDaoImpl`, and `ProductModelDM`. This benchmark models realistic order-processing scenarios: inserting only the order header as a baseline, inserting an order with its line items, and executing a complete workflow that includes saving the order, persisting the contents, and updating inventory quantities.

The benchmarks are parameterized by the number of products per order, with configurations ranging from very small orders to larger ones. Using JMH in average time mode

on top of an in-memory H2 database, the results show an approximately linear scaling of latency with the number of products, confirming the expected $O(n)$ behavior of the workflow. The measurements also indicate that transaction management overhead remains modest, and that the system can process thousands of complete orders per second under the tested conditions.

7.2.3 ProductModelDMBenchmark: Repository Layer. The `ProductModelDMBenchmark` class focuses on the repository layer, and in particular on the `ProductModelDM` class, whose methods are declared `synchronized`. The benchmark exercises three representative operations: retrieving all products (with and without `ORDER BY` clauses), retrieving a single product by primary key, and updating product quantities in storage.

The dataset size and ordering strategy are exposed as JMH parameters, allowing the benchmark to explore different combinations of data volume and sort options. The resulting measurements confirm constant-time behavior for indexed lookups and updates, linear-time behavior for full scans, and an additional overhead when sorting results. The impact of the `synchronized` keyword on performance is negligible compared to database latency, suggesting that the current locking strategy is acceptable for the expected workload.

7.3 Technical Setup

7.3.1 Benchmark Architecture. All benchmarks are launched via a dedicated `BenchmarkRunner` class, which serves as the JMH entry point. The runner executes three benchmark classes (`CartBenchmark`, `OrderProcessingBenchmark`, and `ProductModelDMBenchmark`), each annotated with the appropriate JMH configuration for warmup, measurement, and parameterization. Cart benchmarks run entirely in memory, whereas order-processing and repository benchmarks use an in-memory H2 database to simulate realistic persistence behavior without external dependencies.

To support database-based benchmarks, a shared utility class (`H2TestDatabaseUtil`) encapsulates the setup and teardown logic. It provides methods to create and configure a data source, initialize the schema, populate test data with a configurable number of products, and drop the schema between runs when needed. This centralization ensures that all database benchmarks start from a consistent, isolated state and that changes to the schema or test data model are applied uniformly.

7.3.2 Practical Issues and Fixes. During the development of the JMH suite, several practical issues emerged. For order processing benchmarks, high benchmark throughput could deplete product stock over time, leading to failures in later iterations. This was mitigated by increasing the initial stock levels, adjusting warmup and measurement iterations, and using JMH setup hooks (e.g., `@Setup(Level.Iteration)`) to periodically reset the database state.

In the repository benchmarks, re-initializing the test database for each parameter combination initially triggered duplicate key errors, because the schema was recreated without dropping existing tables. The issue was resolved by explicitly dropping the schema before re-creating it in the setup phase, ensuring a clean database for every parameterized run. Additionally, after refactoring the application code (e.g., changing constructors and dependency injection patterns in `ProductModelDM`), the benchmarks were updated to inject

dependencies via reflection where necessary, restoring compatibility without altering production code.

7.4 Results and Insights

The complete benchmark suite executes 78 parameterized test variations across the three benchmark classes, covering different input sizes, query patterns, and measurement modes, with a 100% success rate. The results provide a clear quantitative separation between in-memory business logic operations and database-backed workflows, as summarized in Tables 7.1–7.3.

Table 7.1. CartBenchmark summary (`Cart.java`, cart sizes $\in \{5, 20, 50\}$, modes: throughput and average time).

Operation	Throughput (ops/s)	Avg. latency (μs/op)	Complexity
`getCartSize`	~780K	~0.001	$O(1)$
`updateQuantity`	~215K	~0.005	$O(1)$
`addProduct`	~34K	~0.026	$O(1)$
`deleteProduct`	10K–25K	0.040–0.107	$O(n)$
`cartCycle` (full)	0.4K–4.6K	0.225–2.683	$O(n)$

Table 7.2. OrderProcessingBenchmark summary (H2 in-memory, `numProducts` $\in \{1, 5, 10\}$, average time mode).

Workflow	Latency (ms/op)	Description
`SaveOrderOnly`	0.034–0.035	Single `INSERT` (baseline)
`SaveOrderWithContents`	0.074–0.341	Order + $n\times$ line-item inserts
`CompleteOrderWorkflow`	0.076–0.363	Order + contents + inventory update

Scaling trend: $\sim O(n)$, with latency increasing by about $\sim 4.8\times$ from 1 to 10 products.

Table 7.3. ProductModelDMBenchmark summary (`ProductModelDM.java`, dataset $\in \{10, 50, 100\}$ rows, average time mode).

Operation	Latency (ms/op)	Complexity and notes
`doRetrieveByKey()`	~0.012	$O(1)$, indexed PK lookup
`doRetrieveAll()` (no sort)	0.014–0.039	$O(n)$, full scan
`doRetrieveAll()` (ORDER BY)	0.017–0.057	$O(n \log n)$, +30–40% overhead
`updateQuantityStorage()`	0.011–0.012	$O(1)$, indexed UPDATE

Cart operations are orders of magnitude faster than repository or workflow operations, while database latency dominates the total time for order processing and product retrieval.

At the same time, the measurements confirm that repository and workflow operations remain within acceptable bounds for an e-commerce application, even under the larger parameter configurations tested. Indexed lookups and updates behave as expected, sorting introduces a predictable overhead, and the use of `synchronized` in `ProductModelDM` does not emerge as a performance bottleneck in the presence of database I/O. These findings validate the current design choices and establish a solid performance baseline for future regressions and optimizations.

7.5 Deliverables and Future Use

The microbenchmarking effort produced a reusable infrastructure consisting of three JMH benchmark classes, a shared database setup utility, and helper scripts to run the suite and post-process the results into machine-readable and human-readable formats. The benchmark results are stored both as raw JMH output and as summarized tables, enabling quick inspection as well as automated comparison in future runs.

This framework can now be integrated into performance regression testing pipelines or reused to benchmark new components as the system evolves. Re-running the benchmarks after significant changes to cart logic, repository methods, or order processing workflows will make it possible to detect performance regressions early and to evaluate the impact of optimizations in a controlled, repeatable way.

8 Dockerization and Container Delivery

8.1 Goal

The goal of this activity was to make the *Vaporant* application easily deployable and reproducible through Docker containers, providing a standardized execution environment for both developers and CI/CD pipelines. More specifically, we aimed to:

(i) automate the build of the application artifact;
(ii) orchestrate the application and the MySQL database via Docker Compose;
(iii) integrate Docker image creation and analysis into the GitHub Actions *Security Pipeline*.

8.2 Scope

Dockerization covered the entire server–side stack, including the Spring Boot WAR and the MySQL instance required by the e-commerce functionalities. We designed a multi-stage `Dockerfile` for building the WAR, a `docker-compose.yml` file to orchestrate application and database, and a dedicated GitHub Actions workflow (*Security Pipeline*) that builds, scans, and publishes the image to Docker Hub.

8.3 Dockerization Plan

The integration plan was structured into three main steps:

- define a multi-stage `Dockerfile` able to compile the project with Maven (JDK 17) and produce a lightweight runtime image based on JRE 17;
- create a Docker Compose configuration that starts the MySQL service and the application container in a coordinated way, including data persistence, initialization scripts, and healthchecks;
- extend the *Security Pipeline* to include image build, Snyk container scanning, and image push to Docker Hub, reusing the same GitHub Secrets already adopted in the security activities.

8.4 Implementation Details

8.4.1 Multi-stage Dockerfile. The `Dockerfile` is organized into two separate stages: **Maven build** and **Java runtime**. In the first stage, the image `maven:3.9-eclipse-temurin-17` is used with `WORKDIR/app`; `pom.xml` is copied first and `mvndependency:go-offline-B` is executed to cache dependency layers, then the `src` directory is copied and `mvncleanpackage-DskipTests-B` is run to produce `vaporant-0.0.1-SNAPSHOT.war`.

The second stage uses `eclipse-temurin:17-jre`, resulting in a smaller runtime image compared to a full JDK. In this stage only the strictly required tools are installed (notably `curl`, used in the healthcheck), the generated `app.war` is copied from the build stage, port 8080 is exposed via `EXPOSE8080`, and an HTTP healthcheck is configured against http://localhost:8080/. The container declares default environment variables for the database connection (`SPRING\_DATASOURCE\_URL`, `SPRING\_DATASOURCE\_USERNAME`, `SPRING\_DATASOURCE\_PASSWORD`), suitable for stand-alone execution and overridden later by Docker Compose in the orchestrated environment. Application startup is encapsulated

in ENTRYPOINT["java","-jar","app.war"], making the container self-contained without external wrapper scripts.

8.4.2 Docker Compose: Application Stack. The docker-compose.yml file defines two main services, mysql and app, attached to the same bridge network vaporant-network and backed by the persistent volume vaporant-mysql-data.

The mysql service uses the mysql:8.0 image with restart policy unless-stopped; credentials and database name are configured through environment variables with default values which can be overridden by external variables. To guarantee data persistence, the volume mysql-data:/var/lib/mysql is mounted in the mysql service. Database schema initialization is not performed via a raw SQL script mounted into docker-entrypoint-initdb.d; instead, it is fully delegated to **Flyway**, which is configured as a Spring Boot dependency (flyway-core and flyway-mysql in pom.xml) and runs automatically at application startup. This approach ensures that schema migrations are version-controlled, repeatable, and consistent across both local and Dockerized environments. A placeholder database/init.sql file is retained in the repository for compatibility, but contains no SQL statements — its comment explicitly states that schema management is handled by Flyway.

The app service is built locally from the project Dockerfile (build:\{context:.,dockerfile:Dockerfile\}) and is exposed on port 8080, with the external port configurable via the APP_PORT variable (default 8080). The environment section of the application service overrides the database connection settings to target the internal mysql service (e.g., jdbc:mysql://mysql:3306/\$\{MYSQL_DATABASE:-storage\}?\ldots) and passes additional configuration parameters such as SPRING_APPLICATION_NAME=vaporant, SERVER_PORT=8080, JSP view prefixes/suffixes, and logging levels for Spring and com.vaporant.

8.4.3 Startup Order, Healthchecks, and Observability. To avoid connection errors during bootstrap, the mysql service defines a healthcheck based on mysqladmin ping with 10-second interval, 5-second timeout, 5 retries, and 30-second start period. The app service declares an explicit dependency through depends_on with condition: service_healthy, ensuring that the application container is started only after the database has been marked healthy by its healthcheck.

In turn, the application service defines an HTTP healthcheck against http://localhost:8080/actuator/health, executed every 30 seconds with a 10-second timeout, 3 retries, and 60-second start period, providing a continuous indication of the logical health of the application rather than just port availability. This health-monitoring setup, combined with the unless-stopped restart policy, improves tolerance to transient faults in both local environments and more complex orchestrated deployments.

8.5 CI/CD Integration and Docker Image Publishing

Dockerization is integrated into the CI/CD pipeline through the *Security Pipeline* workflow (.github/workflows/security.yml), which is triggered in two ways: automatically on successful completion of the main *Build and Analyze* workflow on branch main, and manually via workflow_dispatch.

Within the *Security Scannings* job, after running GitGuardian and Snyk on source code and Maven dependencies, a Docker-specific step builds the container image using `do ckerbuild-tvaporantsd-app.`, which leverages the multi-stage `Dockerfile`. The resulting image is then scanned using `snyk/actions/docker@master` with `--severity-threshold=high`, enabling detection of vulnerabilities in the base layers and system libraries packaged in the container.

Once the scanning phase succeeds, the workflow authenticates to Docker Hub using `docker/login-action@v3` and credentials stored as GitHub Secrets (`DOCKER\_USERNAME`, `DOCKER\_PASSWORD`), and performs tagging and push of the image as `\$\{DOCKER\_US ERNAME\}/vaporantsd-app:latest`. Consequently, each successful run of the Security Pipeline produces a new, security-checked image on Docker Hub, ready for staging or production environments without manual steps.

8.6 Issues and Solutions

During container integration several practical issues emerged, similarly to what happened for the other activities in the project.

A first set of problems concerned credential management: initially, some passwords were present in clear text inside the Docker Compose configuration; these values were removed from version control and replaced with parameterized environment variables, with real values provided via local `.env` files or GitHub Secrets, in line with the remediation strategy described in the GitGuardian section. A second issue was the synchronization between database startup and application startup: without healthchecks, the application could attempt to connect to MySQL while the service was still booting, causing *connection refused* errors. This was solved by introducing the healthcheck on `mysql` and configuring `depends_on: condition: service_healthy` on the `app` service, effectively serializing startup order and reducing failures at boot time.

Finally, adding the HTTP healthcheck in the runtime image required installing `curl`; to avoid bloating the image size, installation is immediately followed by cleanup (`apt-get clean` and removal of package caches), keeping the final image relatively compact while still providing observability on application health.

8.7 Final Results

The resulting configuration allows the entire stack to be started with a single `docker-compose up -d` command, without installing MySQL or Java locally, using the dedicated
`vaporant-network`, the persistent volume `vaporant-mysql-data`, and an initialization script that populates the database with consistent data. In parallel, the Security Pipeline automatically builds, scans, and publishes the `${DOCKER_USERNAME}/vaporantsd-app:latest` image, ensuring that the distributed artifact reflects the most recent and secure state of the codebase and its dependencies.

This container-based infrastructure forms a solid foundation for future extensions towards orchestrators such as Docker Swarm or Kubernetes, and it is already integrated with the quality and security gates described in the previous sections of the report, significantly contributing to the overall dependability of the system.

9 CI/CD and DevSecOps Pipeline

9.1 Goal

The goal of this activity was to implement a reliable CI/CD process for the *VaporantSD* project by automating build, test execution, and quality checks on every relevant code change.

A second goal was to adopt a DevSecOps approach by integrating security scanning and container delivery steps into GitHub Actions, so that security controls are continuously enforced and reproducible.

9.2 Scope

The CI/CD infrastructure is implemented using GitHub Actions and is organized into two workflows: *Build and Analyze* (CI and quality metrics) and *Security Pipeline* (security scanning and container image publication).

The scope includes integration with external services (Codecov, SonarCloud, Snyk, GitGuardian, Docker Hub) configured through GitHub encrypted secrets, ensuring that no tokens or credentials are hard-coded in the repository.

9.3 CI/CD Integration Plan

The integration plan followed three main steps.

(1) First, a CI workflow was defined to build the application, run the test suite, and produce coverage reports, triggered on both `push` and `pull_request` events for the main development branches.
(2) Second, a dedicated static analysis job was added to the same workflow, explicitly depending on the build job and reusing the produced artifacts (compiled classes and JaCoCo reports) to support consistent analysis.
(3) Third, a separate security workflow was introduced and serialized after CI using `workflow_run`, ensuring that security checks and container publication occur only when the upstream CI workflow completes successfully on `main`.

9.4 CI/CD Implementation Details

9.4.1 Build and Analyze: Triggers and Structure. The *Build and Analyze* workflow triggers on `push` and `pull_request` events for branches `main` and `feature/testing-suite`, enabling fast feedback for both direct commits and PR validation.

The workflow defines two jobs: `build` (compilation, testing, coverage, and artifact production) and `sonarcloud` (static analysis), with `sonarcloud` depending on `build` via `needs: build`.

9.4.2 Build job: Build, Tests, Coverage, and Artifacts. The `build` job runs on `ubuntu-latest` and checks out the repository with `fetch-depth: 0`, which is commonly required by quality tools to compute blame/metadata correctly.

It then sets up Java 17 (Temurin) using `actions/setup-java@v4` and enables dependency reuse through `cache: maven`, reducing build time by caching Maven artifacts across runs.

The core verification command is `mvn-Bcleanverifyjacoco:report`, which runs the test suite and generates JaCoCo output, including the XML coverage report at `./target/site/jacoco/jacoco.xml`.

Coverage is uploaded to Codecov via `codecov/codecov-action@v5`, with `fail_ci_if_error: true`, meaning the CI fails if coverage upload fails (i.e., coverage reporting is treated as a gate).

Finally, two artifacts are uploaded for reuse in subsequent jobs: `compiled-classes` from `target/classes` and `jacoco-report` from `target/site/jacoco`, both with `retention-days: 1` to keep storage costs low while still supporting cross-job consumption.

9.4.3 SonarCloud job: Caching, Inputs, and Scan Configuration. The `sonarcloud` job checks out the code and sets up Java 17 again to ensure an isolated execution environment.

To reduce repeated downloads of analyzers, it caches Sonar packages using `actions/cache@v4` with path `\textasciitilde/.sonar/cache` and an OS-scoped key.

It then downloads the artifacts produced by the `build` job (compiled classes and the JaCoCo report), restoring them into the expected Maven paths (`target/classes` and `target/site/jacoco`).

Static analysis is executed via the Maven Sonar scanner plugin with explicit parameters: `-Dsonar.host.url=https://sonarcloud.io`, authentication via `-Dsonar.token=\$\{SONAR\_TOKEN\}`, and project identification via `-Dsonar.projectKey=\$\{SONAR\_PROJECT\_KEY\}` and `-Dsonar.organization=\$\{SONAR\_ORG\}`.

The scan is configured with `-Dsonar.qualitygate.wait=true`, enabling the pipeline to wait for the quality gate evaluation (when the step executes successfully).

The step is marked `continue-on-error: true`, ensuring that transient SonarCloud failures do not block the full workflow outcome while still providing analysis results when available.

9.4.4 Security Pipeline: Serialization, Scanning, and Delivery. The *Security Pipeline* workflow is triggered automatically after completion of *Build and Analyze* on branch `main` and can also be started manually via `workflow_dispatch`.

Its job execution is guarded by an explicit condition: it runs when manually dispatched or when the upstream `workflow_run` concludes with `success`, preventing security scans and publishing from running on failed builds.

The workflow performs GitGuardian secret detection, Snyk code scanning (`snykcodetest--severity-threshold=high`), Snyk Maven dependency scanning with `--severity-threshold=high`, and a Docker image build followed by Snyk container scanning on the locally built image `vaporantsd-app`.

After the security checks succeed, the pipeline authenticates to Docker Hub using `docker/login-action@v3` and credentials stored as GitHub Secrets (`DOCKER\_USERNAME`, `DOCKER\_PASSWORD`).

It then tags and pushes the image as `\$\{DOCKER\_USERNAME\}/vaporantsd-app:latest`, publishing a container artifact only after the full security gate has been executed.

9.5 Issues and Solutions

One key pipeline design issue is preventing downstream security and delivery steps from running on unstable code; this was addressed by separating workflows and serializing them with `workflow_run` plus a success-only guard condition.

Another practical issue is CI execution time and stability when external services are involved; this was mitigated through caching (Maven dependency cache in the build job, Sonar cache in the SonarCloud job) and by making the SonarCloud scan non-blocking (`continue-on-error: true`) while keeping build/test/coverage steps strict.

Finally, all integrations rely on encrypted secrets rather than repository-stored credentials, aligning with the report's broader remediation strategy around secret management and CI/CD security mechanisms.

9.6 Final Results

The final CI workflow automatically builds and tests the project on pushes and pull requests to `main` and `feature/testing-suite`, generates JaCoCo coverage, and uploads coverage to Codecov as part of the standard verification process.

In addition, a gated DevSecOps workflow executes secret detection, code and dependency scanning, container scanning, and then publishes the Docker image to Docker Hub only after a successful upstream CI run on `main`.

Overall, this setup provides continuous quality and security feedback while producing a distributable container artifact through a repeatable and automated process.

9.7 Emerging Best Practices

- Separate CI (fast feedback) from security/delivery (heavier checks), and serialize them with `workflow_run` and success guards to improve signal quality.
- Use caching (`cache: maven` and Sonar cache) to reduce pipeline execution time and increase stability across runs.
- Treat coverage upload as a reliability gate (`fail_ci_if_error: true`) so that quality reporting cannot silently degrade.
- Apply severity thresholds in security tools and extend scanning to the container image to cover runtime dependencies as well.

9.8 Conclusions

The two-workflow GitHub Actions design combines rapid CI feedback with a gated DevSecOps pipeline, improving both development velocity and system dependability.

By continuously enforcing tests, coverage reporting, static analysis, and security scanning before publishing the container image, the pipeline operationalizes the project's dependability goals in a concrete and repeatable manner.